React Fundamental

A JavaScript library for building user interfaces

LEXICON

Objectives

- Prerequisites
- What & Why React
- DOM & React DOM
- React Environment
- JSX
- Components
- Styling React Using CSS
- Variables & Functions

LEXICON

Prerequisites

- Basic knowledge of **HTML**, **CSS** and **JavaScript**
- Basic understanding of the Document Object Model(**DOM**)
- Basic understanding of **ES6** features
  - Let & Const
  - Arrow functions
  - Imports and Exports
  - Classes
- Install **Node.js** (JavaScript runtime environment)
- Basic understanding of how to use **npm**

What is React?

- Open-source JavaScript library for buildig UI
- Created by a software engineer at Facebook in 2011
- Used for handling the view layer (web and mobile apps)
- To create **reusable UI components**
- To update pages without relying on a server-side
- To create sizeable web applications or complex UI

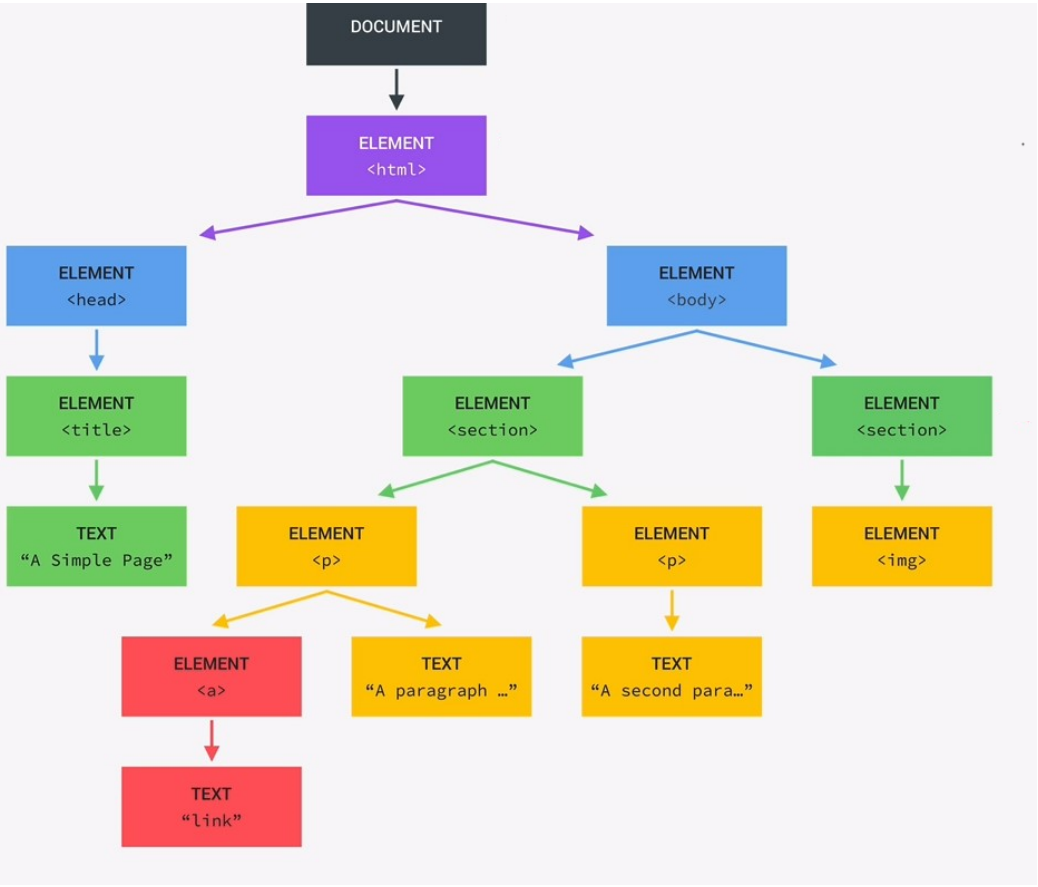
Why use React

1. *Simplicity*
2. *Easy to learn*
3. **Component-based**
4. *Native Approach*
5. ***Performance-Fast***
6. *Testability*

DOM (Document-Object-Model)

- Is a program Interface for web documents (HTML and XML)
- Logical structure of documents
- W3C Standard

```
<html>
  <head>
    <title>A Simple Page</title>
  </head>
  <body>
    <section>
      <p>A paragraph with a <a>link</a></p>
      <p>A second paragraph</p>
    </section>
    <section>
      <img src="dom.png" alt="The DOM" />
    </section>
  </body>
</html>
```



ReactDOM

- DOM is a tree-like structure that contains all the elements and it’s properties
- ReactDOM is a package that provides DOM specific methods like **render(),** findDOMNode() ...
- Used for rendering the components in the DOM
- **render()** method is responsible to display the specific view (HTML code) into the specific element
- To use the ReactDOM in any React app:

```
import { createRoot } from 'react-dom/client';

createRoot(document.getElementById('root')).render(<MyComponent />);
```

React Environment

- There are several ways to setup the react environment
- The simplest way to write **React directly in HTML** using CDN

```
  <head>
      <script src="https://unpkg.com/react@16/umd/react.development.js"></script>
      <script src="https://unpkg.com/react-dom@16/umd/react-dom.development.js"></script>
      <!-- Babel is a JavaScript compiler or transpiler - converts the JSX to standard JS -->
      <script src="https://unpkg.com/babel-standalone@6.26.0/babel.js"></script>
  </head>
<body>
      <div id="root"></div>

  <script type="text/babel">
        class App extends React.Component {
          render() {
            return <h1>Hello world!</h1>
          }
        }
    ReactDOM.render(<App />, document.getElementById('root'))
      </script>
  </body>
```

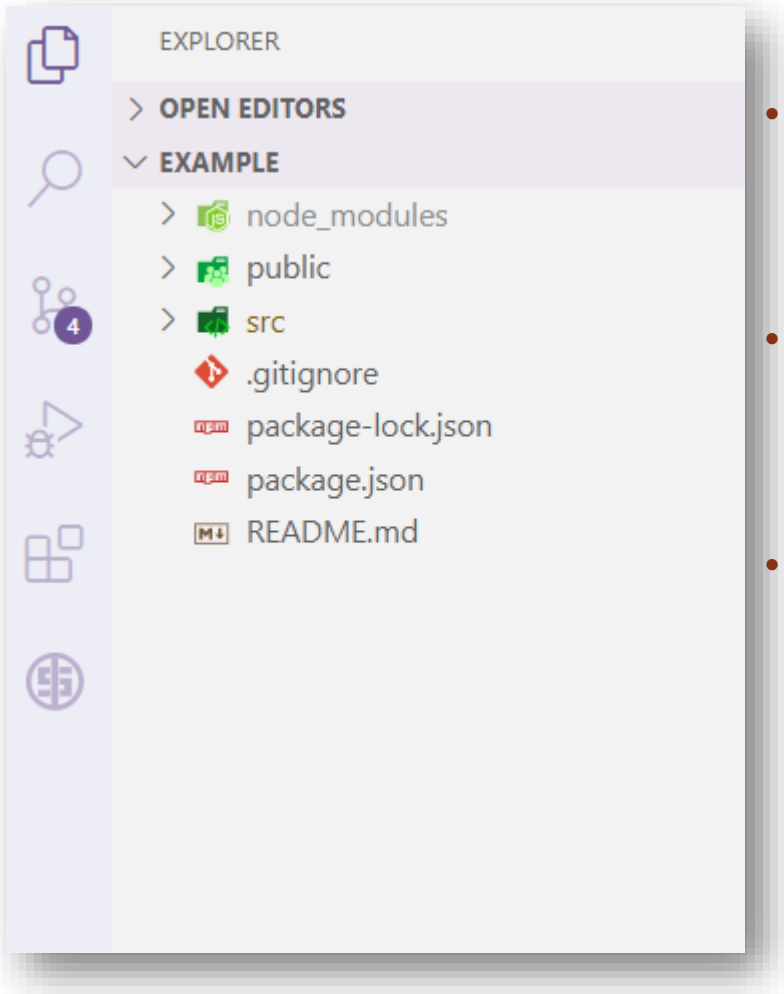
React Environment - CLI

- Use the CLI tool instead CDN
- Comfortable environment for learning React
- Setting up the environment through the *create-react-app* CLI tool
- CLI tool will install React as well as other third-party libraries
- To access the CLI, need to install *Nodejs*
- Nodejs and npm version: *node -v* and *npm -v* in terminal.

```
PS E:\MProject\react\example> npm -v
6.14.8
PS E:\MProject\react\example> node -v
v14.15.0
```

```
C:\Users\Your Name> npx create-react-app test-react-app
```

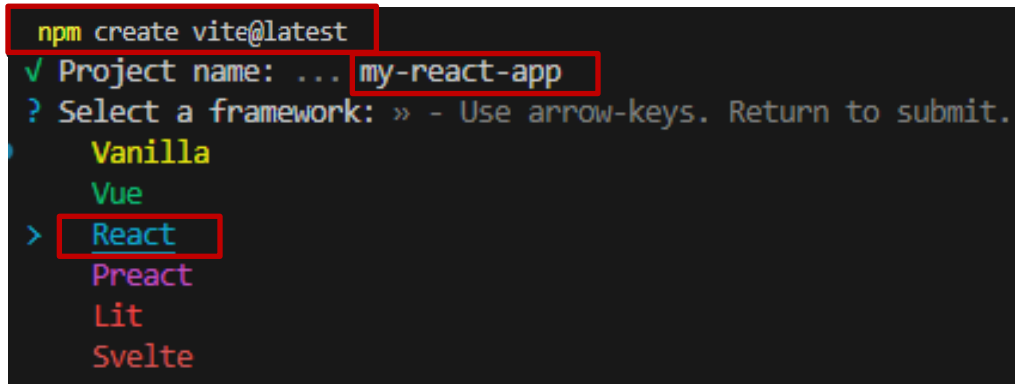
Understand the Folder Structure

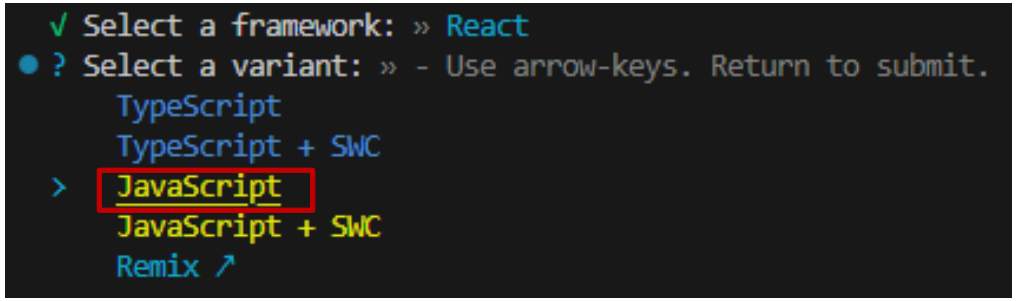


- **/node_modules**:
  - Third-party and React libraries.
  - Packages that are later installed through **npm**.
- **/public**:
  - Public asset of the application
  - **Static** data
- **/src:**
  - The React components
  - External CSS files
  - **Dynamic** assets

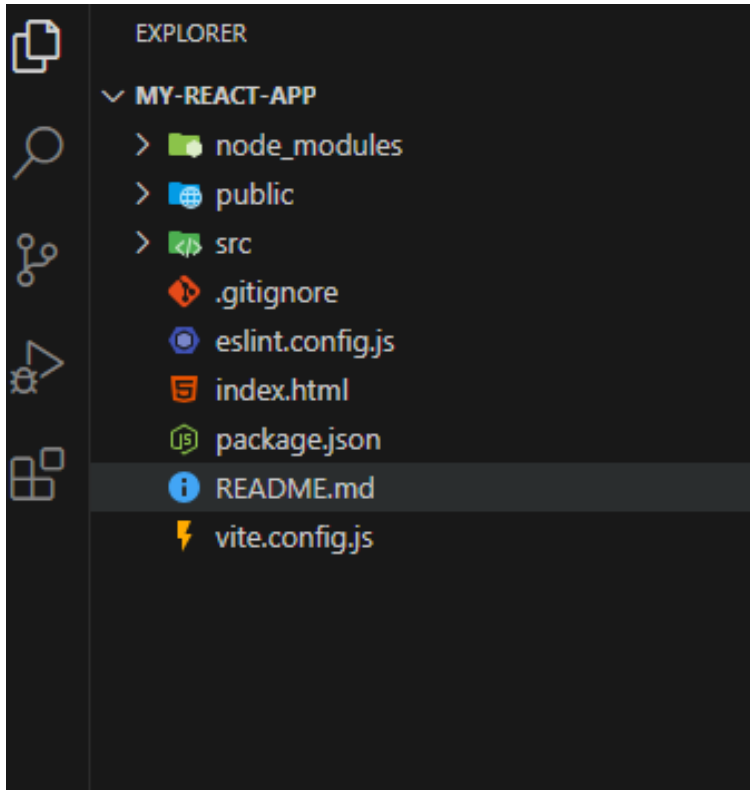
React Environment - Vite

- Use Vite for faster builds and better performance
- **Comfortable and efficient environment** for learning React and front-end development
- Setting up the environment through the *Vite CLI* tool
- **Vite automatically configures** React and optimizes performance during development
- To use Vite, need to install *Nodejs* and access it through the terminal
- To create a Vite React project:





Understand the Folder Structure



- **/node_modules**:
  - Third-party and React libraries.
  - Packages that are later installed through **npm**.
- **/public**:
  - Public asset of the application
  - **Static** data
- **/src:**
  - The React components
  - External CSS files
  - **Dynamic** assets

vite.config.js: The configuration file for Vite.

JSX

- JSX stands for JavaScript XML or JavaScript Syntax extension
- Allows developer to write HTML in React
- Makes easier to write and add HTML in React
- Provides module-based and reusable code packages**(components)**.
- JSX codes are compiled by Babel (a javascript compiler)
- JSX provides better error messages
- JSX increases code readability

JS Code

```
const App = () => {
        return React.createElement("div", null
, "Hello world!");
      };
```

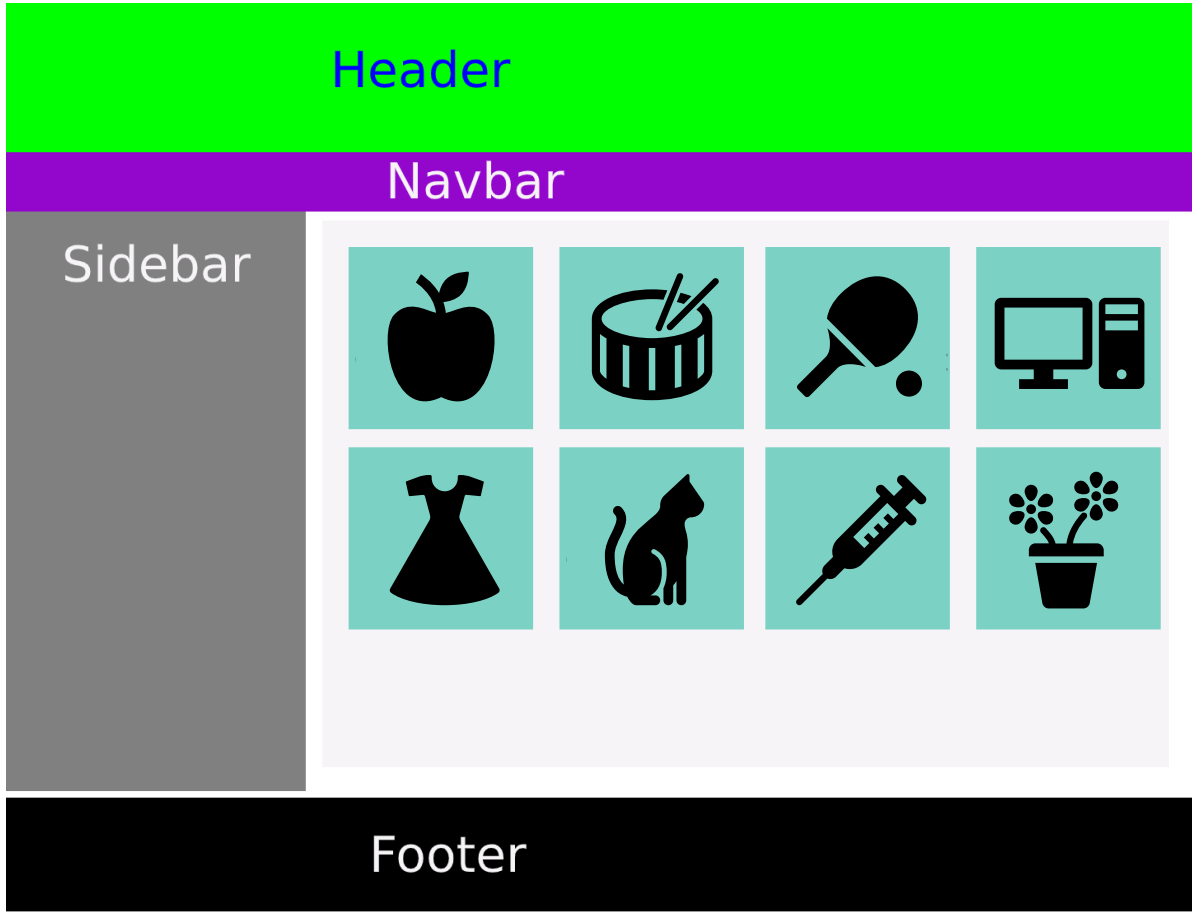
JSX Code

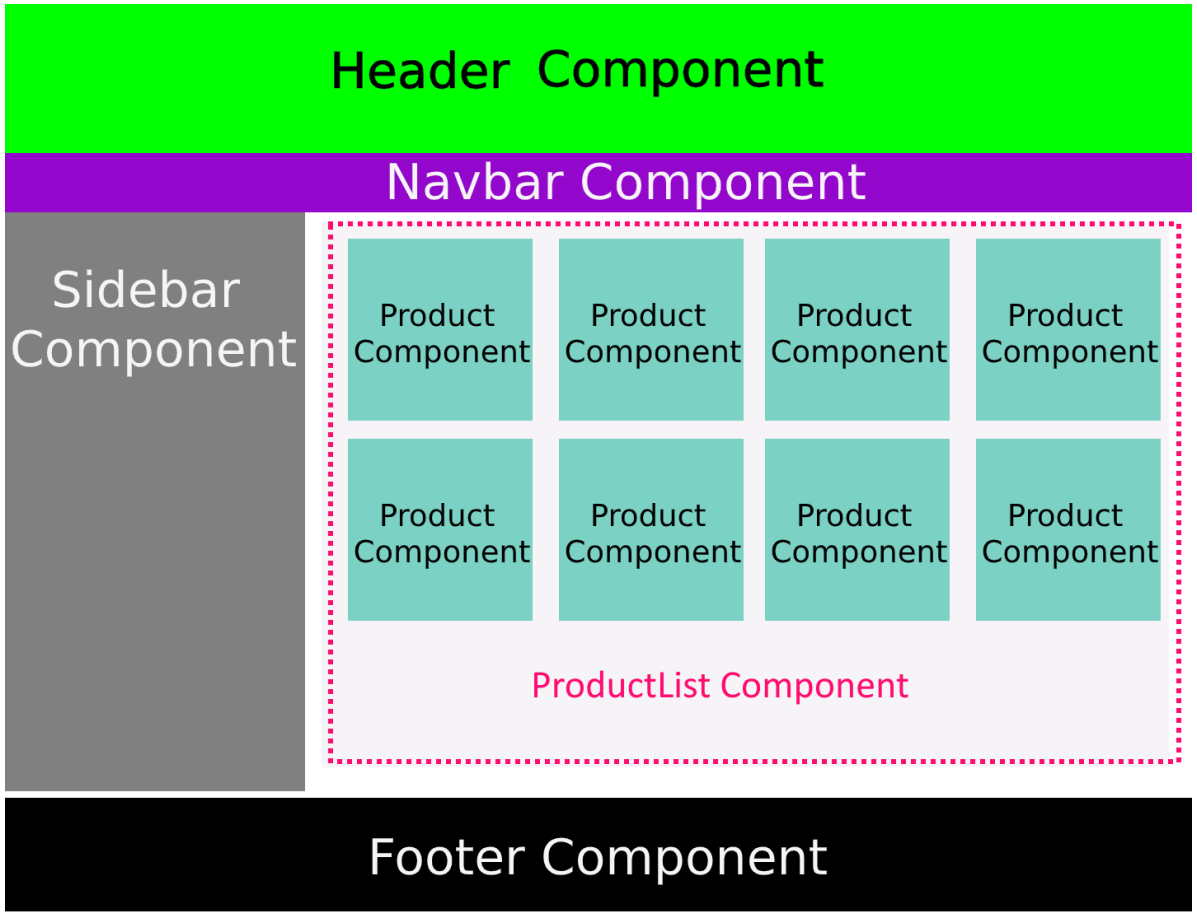
```
const App = () => {
        return <div>Hello world!</div>
    }
```

What are React Components?

- All User Interfaces in React are Component
- Component is one of the **basic building blocks** of React
- Is a JavaScript class or function that takes inputs and renders the HTML elements
- Class and **Function** components
- Components are independent and reusable
- They are working with JS functions and return the various HTML elements
- Component's name must start with a capital letter

What are React Components?





Class Component

- A class component requires to **extends** from **Component**
- Has local state / stateful
- Has **render** function that returns a React element

```
import React, { Component } from 'react';

class App extends Component {
    render() {
        return (
            <div>
                <h1>Hello React!</h1>
            </div>
        );
    }
}
export default App;
```

```
import ReactDOM from "react-dom";
import App from "./App";

ReactDOM.render(<App />, document.getElementById('root'));
```

Functional Component

- Is just a plain JavaScript function
- Accepts **props** as an argument
- Returns a React element
- Common Components
- Simple, Readable & Reusable

```
function Welcome(props) {
    return <h1>Hello, {props.name}</h1>;
  }
  function App() {
    return (
      <div>
        <Welcome name="Mehrdad" />
        <Welcome name="Ulf" />
        <Welcome name="Simon" />
      </div>
    );}
```

```
function Welcome() {
    return <h1>Welcome to my React App</h1>;
  }
  const element = <Welcome />;
  ReactDOM.render(element, document.getElementById('root'));
```

Arrow Function

- A component can be created as a modern (ES6)
- Easy to read
- Skip the return keyword
- Less code
- More readable

Note:

- All functional components can be called Stateless Function Component. It means they just take an input as props and return an output as JSX.
- React Hooks made it possible to use state in Function Components through **useState** React Hook Function.

```
const Header = () => "Welcome To My App";
const MenuList = () => {
  return (
    <ul>
      <li>
        <Header />
      </li>
      <li>Home</li>
      <li>Services</li>
      <li>Contact Us</li>
    </ul>
  );
};
```

Break down a Table to React Components

- A Component is one of the **core building blocks** of React
- Components are independent & reusable

Table

TableHeader

TableRow

TableAction

Naming convention

- JSX is not HTML but actually closer to JavaScript, so there are a few key differences to note when writing it.
- `className` is used instead of `class` for adding CSS classes, as `class` is a reserved keyword in JavaScript.
- Properties and methods in JSX are camelCase – `onclick` will become `onClick`.
- Self-closing tags must end in a slash - e.g. `<img />`
- Component can only return one parent element but that element can have many child elements.

```
export const App = () => {
    return (
        <div className="App">
            Hello world
        </div>
    )
}
```

```
class App extends Component {
   render() {
      return (
         <div className="App">
            Hello world
         </div>
      )
}}
```

Styling Using CSS

- Inline Styling
- CSS Stylesheet
- **CSS Modules**

App.css

```
h1 {
    background-color: #282c34;
    color: white;
    padding: 40px;
    font-family: Arial;
    text-align: center;
  }

  .test {
    color: rgb(0, 192, 48);
    font-family: Arial;
    font-size: 28px;
  }
```

```
const App = () => {
  return (
    <div>
      <h3>Styling Using CSS</h3>
      {/* Inline Styling */}
      <h1 style={{ color: "red" }}>Inline Styling!</h1>
      {/* Use backgroundColor instead of background-color: */}
      <h1 style={{ backgroundColor: "lightblue" }}>Inline Styling!</h1>
    </div>
  );
}
```

```
import './App.css';
```

```
{/* CSS Stylesheet */}
<p className={'test'} >Add a little style!</p>
```

Styling Using CSS Modules

- Convenient and available only for the component that imported
- Create the CSS module with the .module.css extension

```
import ReactDOM from "react-dom";
import styles from './app.module.css';

const App = () => {
  return (
    <div>
      <h1>Hello Style!</h1>
      <p className={styles.test} >Add a little style!</p>
    </div>
  );
};

ReactDOM.render(<App />, document.getElementById("root"));
```

app.module.css

```
h1 {
    background-color: #282c34;
    color: white;
    padding: 40px;
    font-family: Arial;
    text-align: center;
  }

  .test {
    color: rgb(0, 192, 48);
    font-family: Arial;
    font-size: 28px;
  }
```

Variables & Functions

- JavaScript can be used inside JSX syntax if they are enclosed by **curly braces {}**.

```
 const names = ["Mehrdad Javan","Ulf Bengtsson ","Simon Elbrink","Erik Svensson"];
 const getLastName = (name) => {
   let lastName = name.split(' ')[1];
   return <span>{lastName}</span>
 }
export const NameList = () => (
  <ul>
    {names.map(name => (
      <li>
        <div>{name} - {name.toUpperCase()} - {getLastName(name)}</div>
      </li>
    ))}
  </ul>
);
```

Links

https://codepen.io/

https://en.wikipedia.org/wiki/Document_Object_Model

https://www.javascripttutorial.net/es6/

https://react.dev/

https://www.taniarascia.com/getting-started-with-react/